

AgriChain — System Handoff

A complete technical reference for the AgriChain platform: every module that exists, how the pieces feed one another, the reasoning behind the design decisions, and an honest account of what is still missing.

Generated 4 October 2026 · Repository `/Users/kshitijdeshmukh/AgriChain` · branch `main`
Verified against a live stack: 289/289 backend tests passing, frontend production build clean (573 modules).

HOW TO USE THIS DOCUMENT

This is written to be handed to another engineer or AI assistant with *no prior context*. Every claim here was checked against the running code, the live database, or a live HTTP call — not inferred from file names. All paths are repository-relative.

Status labels mean exactly this: **WORKING** verified end to end · **NEEDS PROCESS** code is complete but depends on a service that is not currently running · **GAP** missing, stubbed, or dead code.

Contents

1. What AgriChain is — the central idea
2. System topology — four processes and a database
3. The six principles that explain the code
4. Module reference
 1. Authentication & roles
 2. Farms & field boundaries
 3. NDVI & satellite imagery
 4. Disease detection (CNN)
 5. Weather intelligence
 6. Spoilage risk
 7. Market intelligence — the mandi channel
 8. Transport & freight
 9. Buyer marketplace — the direct channel
5. How it all connects — the dependency graph
6. Data model — 28 tables
7. Complete API surface
8. Verified status right now
9. What is remaining
10. Running the stack
11. Glossary

1. What AgriChain is — the central idea

AgriChain is a decision-support platform for Indian smallholder farmers. It answers one question across every module: **what should I do with this crop, and where will it leave me the most money?**

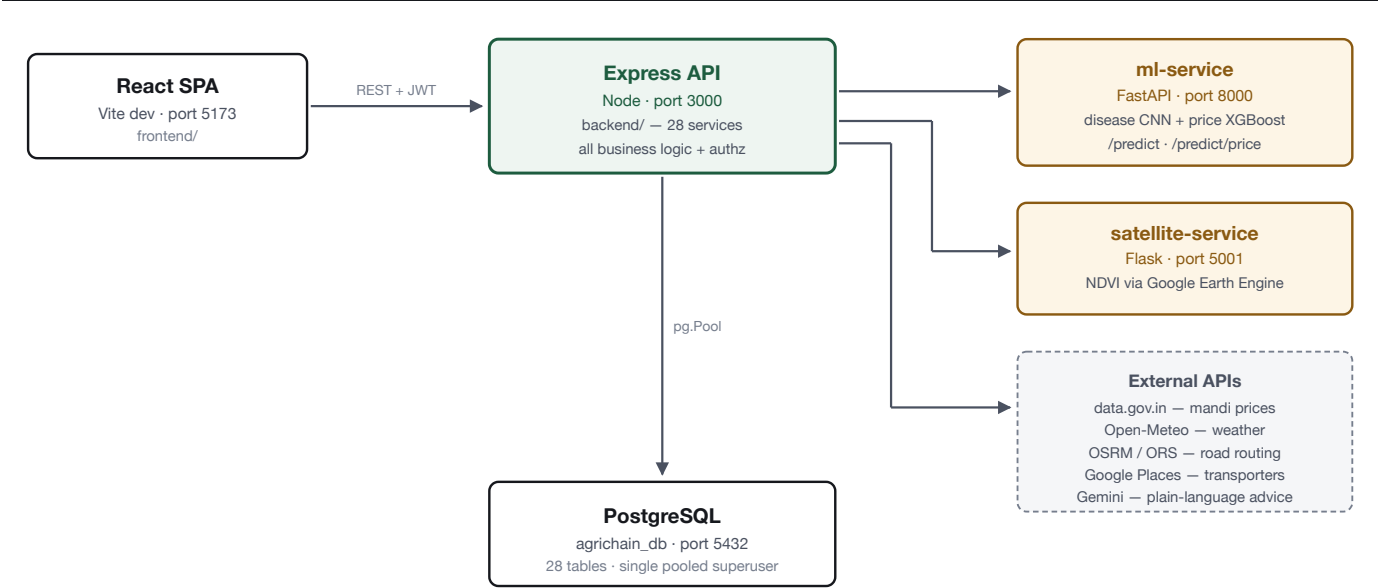
The platform's organising insight is that **the advertised price is not the farmer's income**. A mandi quoting ₹31/kg 135 km away can pay less than one quoting ₹28/kg 16 km away, once freight, commission, cess, hamali and the share of the load that rots in transit are subtracted. Every ranking in this system is therefore ordered by **expected money in hand**, never by board price, distance, or any single input.

Two selling channels are modelled, and the architecture's main achievement is that they are **comparable on identical terms**:

CHANNEL	WHAT IT IS	HOW THE FARMER REACHES IT
APMC mandi	Government-regulated wholesale markets with published daily prices	Market Intelligence module ranks nearby mandis by expected money
Direct buyer	Private wholesalers, processors, exporters who post what they need	Buyer Marketplace — negotiate a price, agree a deal that reserves the crop

Because both channels are costed by the *same* engine (`netReturnService`) using the *same* spoilage model, routing, and freight rates, the endpoint `GET /api/marketplace/selling-options` can place a mandi and a private buyer side by side and the comparison is meaningful rather than apples-to-oranges. This is the single most important structural fact about the codebase.

2. System topology — four processes and a database



Process topology. The browser only ever talks to Express — no API keys and no third-party calls reach the client. Both Python services and all five external APIs are optional at runtime.

PROCESS	PORT	STACK	STATUS NOW	NOTES
Express API backend/	3000	Node 18+, Express 4	WORKING	All business logic, all authorization. The only process that touches the database.
React SPA frontend/	5173	React 18, Vite 5, Tailwind	WORKING	21 pages. Production build clean.
ml-service ml-service/	8000	FastAPI, TensorFlow, XGBoost	PORT CONFLICT	Port 8000 is currently held by an unrelated "LLM Gateway" process. See §8.
satellite-service satellite-service/	5001	Flask, Google Earth Engine	NOT RUNNING	NDVI returns an explicit error when absent; nothing else is affected.
PostgreSQL	5432	Postgres + pg.Pool	WORKING	28 tables. Schema auto-applies on boot (idempotent).

3. The six principles that explain the code

Reading these first makes the rest of the codebase predictable. They are applied consistently and are the "why" behind most decisions an outsider would otherwise find surprising.

1 · Rank by expected money, never by one input

`backend/src/services/netReturnService.js` — called the "heart of AgriChain" in its own header comment. Every other engine exists to feed it:

gross sale value	price x the whole load
- spoilage loss value	the part that never gets sold
= expected sale value	what the buyer actually pays for
- transport cost	getting it there (and the truck back)
- other selling costs	commission, cess, hamali, weighing
= EXPECTED MONEY	what the farmer walks home with

The engine returns a full itemised ledger, so every deduction shown to a farmer traces to a number this function produced. It is deterministic — no ML, no randomness, same inputs, same rupees.

2 · Label the provenance of every number

Engines declare what they are, so an API response can never imply more rigour than exists. Examples found in the code:

- `RULE_BASED_BASELINE` — the spoilage engine is a physics-based rule set, not a fitted model. Its header explains why: there is no labelled Indian post-harvest spoilage dataset in the repo, and *"fabricating one and fitting a regressor to it would manufacture an accuracy figure that means nothing."*
- `STRAIGHT_LINE` vs a real route — a haversine estimate is never presented as a road distance.
- `DEMO_SEED` vs `MANDI_API` — every price row records its source, and demo rows carry `is_demo_data = TRUE`.
- `is_demo_trained: true` — the price model's own metadata admits it was trained on seeded data.

3 · Integer paise, never floats

`backend/src/utils/money.js`. Rupees appear only at API boundaries. The stated reason is precise: *"a cascade of eight float operations across a dozen markets drifts, and drift here reorders the ranking — it changes which mandi a farmer drives to."* Offer totals are **stored, not derived**, so an agreed figure can never be recomputed differently later.

4 · Authorization in the service layer, not the database

WHY THERE IS NO ROW LEVEL SECURITY

Everything connects through one pooled `postgres_superuser` (`backend/src/config/db.js`), and PostgreSQL superusers **bypass RLS entirely** — policies would be decorative. Real RLS needs per-request database roles and `SET LOCAL` session context, a re-architecture of the connection layer affecting all nine modules.

Used instead: **ownership checks in every service** (no read or write path skips them) plus database **CHECK**, **FOREIGN KEY** and **UNIQUE constraints**, which Postgres enforces regardless of who connects — including a superuser. Those constraints are what make over-reservation and double-acceptance structurally impossible. This is the most significant known gap; migration path is documented in `docs/Marketplace/README.md §1`.

A corollary applied everywhere: **identity is never taken from the request body**. `req.user.id` comes from a verified JWT and `req.buyerProfile` from middleware; a `userId`, `farmerId` or `buyerId` in a body is ignored.

5 · Degradate with a flag, never with a guess

From `marketService`'s header: only two conditions are fatal — no farm coordinates, and no price data for the crop anywhere. Everything else continues with an explicit flag (`pricePredictionAvailable:false`,

`weatherAvailable: false` , routing falling back to a labelled straight line). *"A farmer gets a usable, honestly-labelled answer or a clear error, never a confident-looking guess."*

6 • Never ask for data the system already holds

`availabilityService.suggestFromFarms` pre-fills a crop listing from the farmer's saved fields — crop type, coordinates, farm name — leaving only the one genuinely new fact: how many kilograms. The `farms` table has no quantity column, which is precisely why `farmer_crop_availability` exists.

4. Module reference

4.1 Authentication & roles WORKING

backend/src/controllers/authController.js · services/roleService.js

JWT bearer tokens, 7-day expiry, bcrypt password hashing. Three roles: `farmer` is **implicit** for every account; `buyer` and `admin` must be explicitly granted (`user_roles` table). Registering a business via `POST /api/buyers/profile` grants the buyer role.

Four middleware variants exist, and the distinction matters:

MIDDLEWARE	BEHAVIOUR AND WHY
<code>authMiddleware</code>	Requires a valid JWT. Rejects otherwise.
<code>optionalAuthMiddleware</code>	Populates <code>req.user</code> if a token is present, else continues. Lets anonymous callers run a disease prediction.
<code>optionalBuyerMiddleware</code>	Attaches <code>req.buyerProfile</code> if the caller happens to be a buyer, without requiring it. Needed where being a buyer changes <i>what</i> you see but is not a precondition — e.g. browsing requirements. <code>requireBuyer()</code> would lock farmers out.
<code>roleMiddleware</code>	<code>requireBuyer()</code> (also rejects suspended buyers), <code>requireAdmin()</code> .

Note: the whole marketplace uses strict `authMiddleware`, never optional auth, because it holds private B2B conversations and negotiated prices. Elsewhere an unauthenticated caller may resolve to a development user id — harmless for reading your own NDVI history, unacceptable here.

4.2 Farms & field boundaries WORKING

backend/src/controllers/farmController.js · database/schemas/farms.sql

CRUD for farms with a name, point location, area in hectares, crop type, and a GeoJSON-style `boundary_coordinates` (JSONB) polygon drawn on a Google map in the frontend. Farm coordinates are the entry point for nearly everything else: market candidate search, routing, spoilage transit estimation, NDVI, and weather all start from a farm's latitude/longitude.

Frontend: `FarmSetup`, `FarmBoundarySetup`, `SavedFields`, `FarmView`, `FieldAnalytics`.

4.3 NDVI & satellite imagery NEEDS PROCESS

backend/src/services/ndviService.js · ndviStorageService.js · jobs/ndviRefreshJob.js · satellite-service/

NDVI (Normalised Difference Vegetation Index) measures crop vigour from Sentinel-2 bands via Google Earth Engine. The Flask service at port 5001 does the Earth Engine work — cloud masking, band math, optional per-pixel grids — and Express stores results in `ndvi` and `ndvi_history`.

A **daily refresh job** (`ndviRefreshJob`) walks every farm with a boundary on a schedule set by `NDVI_DAILY_REFRESH_*` env vars, and can alert on a vigour drop (`NDVI_ALERT_DROP_ABS`). A unique index on (`field_id`, `captured_date`) prevents duplicate history rows.

Current state: the satellite service is not running, so the startup refresh logs `Satellite service is not available` at `http://localhost:5001` and fails for each farm (7 farms, 0 succeeded in the last observed run). This is a *missing process*, not a code defect — the error is explicit and no other module is affected. Start the Flask service to restore it; it additionally needs Google Earth Engine credentials.

4.4 Disease detection NEEDS PROCESS

ml-service/ · backend/src/services/mlService.js · docs/DiseaseDetection/

A trained **EfficientNetB0** CNN classifies leaf photographs into 12 classes across three crops. The model artifact is committed (`disease_model.keras` , 35 MB) with full provenance metadata:

PROPERTY	VALUE
Architecture	EfficientNetB0, 224×224×3 input
Classes	12 — Cotton (healthy, bacterial blight, alternaria leaf spot, leaf curl virus), Soybean (healthy, rust, bacterial pustule, brown spot), Orange (healthy, citrus canker, black spot, greening)
Test accuracy	98.58% · macro-F1 0.9365 · weighted-F1 0.9858
Traceability	SHA-256 of the artifact, and a <code>split_fingerprint</code> tying it to an exact train/test split

The pipeline adds severity scoring (`ml-service/src/severity.py`) and a **combined advisory** (`POST /api/disease/combined-advisory`) that merges disease findings with weather-derived risk — an example of cross-module composition. Detections persist to the `diseases` table with the uploaded image served through `/uploads` . The dataset work is documented across 17 reports in `dataset/reports/`, including near-duplicate calibration and leakage recalibration.

Current state: `GET /api/disease/health` returns `{backend: "healthy", ml_service: "unavailable"}` — see the port conflict in §8.

4.5 Weather intelligence WORKING

`backend/src/services/weatherService.js`

Open-Meteo — free, no API key, which is why it was chosen. Ten-minute in-memory cache. Serves current conditions, multi-day and hourly forecasts, alerts, farming recommendations, and a **disease-risk** reading derived from temperature and humidity that the disease module consumes for its combined advisory. Also feeds `marketService` , where ambient conditions adjust the spoilage estimate.

Verified live: `{"status":"operational","provider":"Open-Meteo"}` .

4.6 Spoilage risk WORKING

`backend/src/services/spoilageService.js` · `database/schemas/spoilage.sql`

Deterministic, external-dependency-free, and deliberately simple:

```
effectiveShelfLife = baseShelfLife x storageFactor x tempFactor x humidityFactor
exposure           = daysSinceHarvest + transitDays
riskScore           = (exposure / effectiveShelfLife) x 100
```

`tempFactor` uses a **Q10 respiration model** (respiration roughly doubles per 10 °C above the crop's optimum) — the standard first-order approximation for post-harvest deterioration, and auditable by an agronomist.

This module also owns the **crop catalogue**: 18 crops with perishability class, base shelf life, optimal temperature and humidity, and a grain flag. `getCropProfile()` is consumed by six other services, making it the most depended-upon function in the backend. The catalogue is exposed at `GET /api/crops` and is what the frontend crop pickers read, so a user cannot submit an unsupported crop.

Verified live: `{"status":"operational","supportedCrops":18}` .

4.7 Market intelligence — the mandi channel WORKING

`backend/src/services/marketService.js` and 9 collaborators · `docs/MarketIntelligence/`

This is the largest subsystem. `POST /api/market/recommend` orchestrates a ten-stage pipeline:

```
farm location -> candidate markets -> observed prices -> price forecast
-> road routing -> transport cost -> weather -> spoilage risk
-> net realizable return -> ranking -> sell now / wait
```

Each stage is its own service; `marketService` only sequences them and prevents one failing stage from sinking the request.

PRICE DATA

`marketPriceIngestService` pulls real mandi prices from **data.gov.in** via a provider abstraction (`services/providers/`), with `commodity_provider_map` and `market_provider_map` translating between their vocabulary and ours. Runs are audited in `price_ingest_runs`. Every price row carries a source and a freshness classification (`FRESH` / `RECENT` / `STALE`) derived from configurable day thresholds.

PRICE FORECASTING

XGBoost regressors at 1-day and 3-day horizons (`ml-service/models/price/`), predicting the *delta* from the last known price and reconstructing the level as `lag_1 + predicted_delta`. Test MAE ₹40.40, MAPE 1.82%.

Honest caveat, from the model's own metadata: `trained_on_real_data: false, is_demo_trained: true`, trained on 18,560 `DEMO_SEED` observations across 5 crops and 16 markets. It beats a persistence baseline by only **0.52% MAE** — i.e. barely better than "tomorrow's price equals today's". Treat the forecast as scaffolding awaiting real training data, not as a working predictor. The recommendation does not depend on it: it informs sell-now-vs-wait only.

SELL NOW OR WAIT

`sellTimingService` compares expected money today against expected money after waiting — factoring the forecast *and* the extra spoilage the delay causes. It only advises waiting if the gain clears both `SELL_WAIT_MIN_GAIN_PERCENT` and `SELL_WAIT_MIN_GAIN_RUPEES`, so it cannot recommend waiting for a trivial gain.

PLAIN-LANGUAGE EXPLANATION

`marketExplanationService` asks Gemini to narrate a recommendation in the farmer's language. Critically, **the narration cannot change any figure** — the backend enforces that — so it is safe to render beside the numbers.

Verified live: 16 markets, real `MANDI_API` price data for 14 crops plus `DEMO_SEED` for 5.

4.8 Transport & freight WORKING

```
services/transportService.js · transportCostService.js · freightRateService.js · fuelPriceService.js ·
routingService.js · googlePlacesService.js
```

Four layers, each independently useful:

- **Routing** — OSRM by default, OpenRouteService optional. On failure it falls back to haversine × a 1.3 detour factor (the commonly cited circuitry factor for Indian district roads) and *labels the result* `STRAIGHT_LINE_ESTIMATE`. Cached, with a configurable TTL.
- **Cost** — per-km rates from `transport_config`, adjusted by live diesel price (`fuel_prices`), including the empty return leg. Cost per kg falls as the load grows, since freight is shared — a property under unit test.
- **Freight quotes** — real operator quotes in `transport_rate_quotes`, with a max-age guard so a stale quote is not silently reused.
- **Transporter discovery** — Google Places (New) `searchText` finds hauliers near a recommended mandi. Called **server-side only**, so the API key never reaches the browser.

Verified live: `{"status":"ok","googlePlaces":{"configured":true,"enabled":true}}`.

4.9 Buyer marketplace — the direct channel WORKING

controllers/marketplaceController.js · marketplaceChatController.js · buyerController.js · 8 services · docs/Marketplace/README.md

The second selling channel, and the most recently completed. A buyer posts a requirement; matching farmers discover it, compare buyers by money actually kept, chat privately, negotiate with structured offers, and agree a deal that reserves the crop.

THE FOUR-BUCKET QUANTITY MODEL

`farmer_crop_availability` tracks quantity in four columns, and `availabilityService` is the only module permitted to move stock between them:

COLUMN	MEANING
<code>total_harvested_kg</code>	What came off the field
<code>available_kg</code>	Offered to buyers right now
<code>reserved_kg</code>	Locked by an accepted deal, not yet handed over
<code>sold_kg</code>	Handed over, deal completed

Why: collapsing these into one "quantity" column is how a marketplace sells the same 500 kg twice. A database `CHECK` (`available + reserved + sold <= total_harvested`) is the backstop, and `reserve()` is the only path from available to reserved — taking a row lock inside the caller's transaction.

MATCHING ENGINE

`matchingService.evaluateMatch` is a **pure function** over two loaded records — no I/O — which makes every eligibility rule unit-testable in isolation. It returns a verdict with **plain-language reasons, not a score** ("Same crop (Tomato)", "Your 400 kg covers what they still need"), plus named exclusion codes when ineligible. Two directions are served:

- `findMatchesForFarmer` — "buyers looking for your crop", ordered by advertised value
- `findFarmersForRequirement` — "who can fill this requirement", ordered *nearest first*, because for a buyer a nearer load is cheaper and arrives fresher

An unrecognised quality grade is treated as "unknown" rather than "fails" — *excluding a farmer because they typed "Premium" would lose them a real sale*.

EXPECTED-MONEY COMPARISON

`buyerComparisonService` is where the two channels meet. It depends on seven other services and powers:

- `GET /api/marketplace/farmer/top-buyers` — buyers ranked by money in hand, not advertised price
- `GET /api/marketplace/selling-options` — **mandis and direct buyers in one comparison**

The demo scenario exists to prove the point: Buyer A advertises ₹31/kg but is 135 km away with the farmer paying freight; Buyer B offers ₹28/kg, is 16 km away and collects at their own cost. B leaves the farmer more money, and the UI ranks B first and says why.

OFFERS, DEALS AND CONVERSATIONS

Offers are structured proposals (`quantityKg`, `pricePerKg`, `deliveryTerms`, `expiry`) that can be accepted, rejected, countered or withdrawn. Acceptance is transactional: it reserves crop, decrements the requirement's remaining quantity, and creates a deal — with openness re-checked *inside* the transaction, not just before it. Counter-offers chain via `parent_offer_id`, preserving the negotiation history.

Every conversation is anchored to a requirement. A `UNIQUE` (`requirement_id`, `farmer_user_id`, `buyer_user_id`) constraint with `ON CONFLICT` prevents duplicate threads. Chat is **polled** every 6 s while a thread is open and the tab is

visible — not pushed. Attachments are served only through an authorising route that verifies participation before streaming bytes; `express.static` is deliberately not used for that directory, so guessing a filename gets a caller nothing.

PRIVACY POSTURE

- Buyer phone and email are **never** sent to farmers. `buyerService.toPublic()` is the single place that decides this.
- Farm coordinates are never sent to buyers. The browse endpoint computes distance **server-side** and returns only the number — verified by assertion in testing.
- Verification status is backend-controlled. A buyer can submit for review; only an administrator can grant it. Attempting to `PATCH` it is rejected — confirmed by live test.

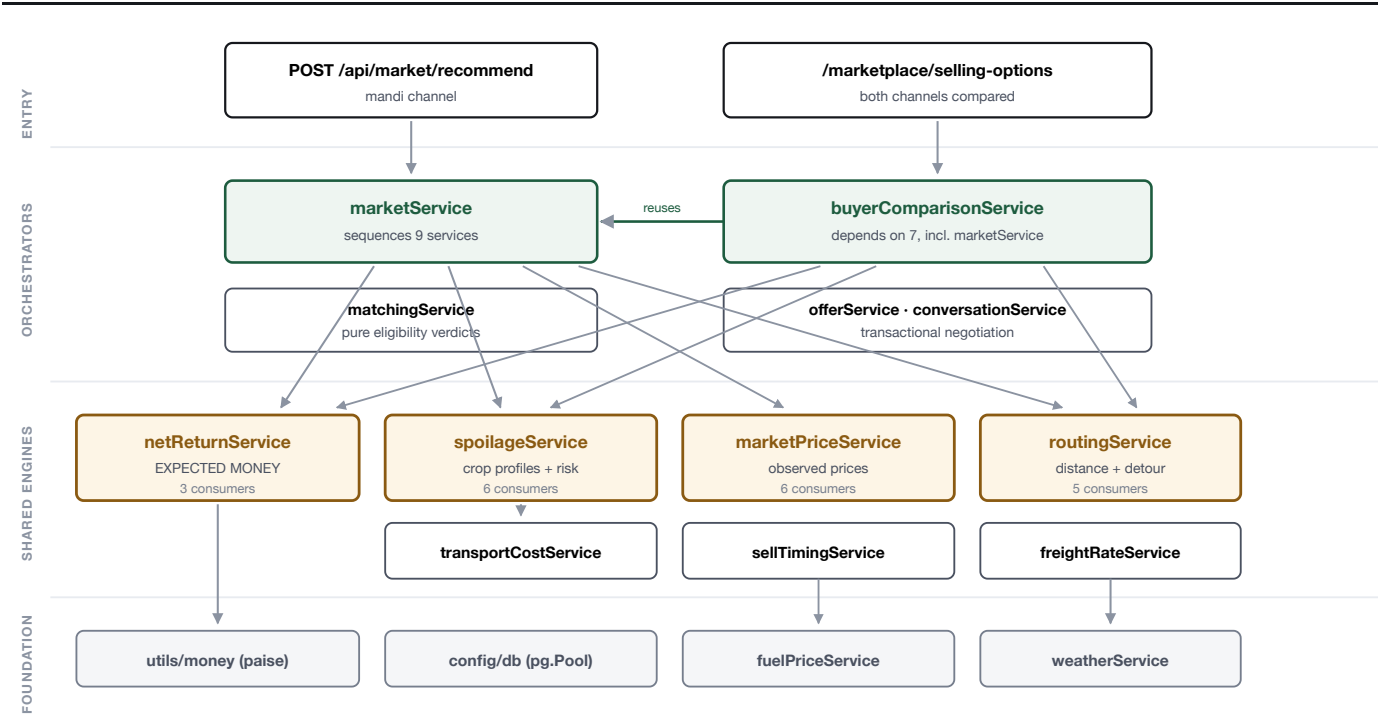
RECENTLY COMPLETED WORK

The buyer-facing half of this module was finished in the most recent development pass. Before it, only posting a requirement worked. What changed:

AREA	WHAT WAS WRONG, AND THE FIX
Browse Farmers	The page was a hard-coded "coming soon" stub and no endpoint existed. Added <code>GET /api/marketplace/availability</code> (<code>availabilityService.browse</code>) with a separate privacy-safe projection, plus crop/quantity/distance/grade filters.
Matching farmers	The endpoint worked but nothing ever called it. New <code>MyRequirements</code> page surfaces matches per requirement with the engine's reasons, and shows exclusion diagnostics when empty.
Buyer-initiated contact	The backend supported it (<code>senderIsBuyer</code>) but there was no UI entry point anywhere. Added shared <code>Contact</code> and <code>Offer</code> dialogs.
Dead routes	<code>/marketplace/requirements/:id</code> was referenced from two places but was never a route in <code>App.jsx</code> — both clicks went nowhere. Repointed to <code>/buyer/requirements/:id</code> , gated on <code>mySide === 'buyer'</code> .
Unread badges	<code>markConversationRead</code> was never called, so counts only ever went up. Now cleared on open and on poll.
Other	Requirement publish/edit wired; <code>/buyer/requirements</code> no longer a duplicate dashboard; block/unblock and report wired; profile editing added; free-text crop box replaced with a picker from <code>GET /api/crops</code> .

One design constraint is worth carrying forward: **a buyer's offer or conversation must be anchored to one of their own open requirements for that crop**. This is enforced in `conversationService.findOrCreate` and `offerService.resolveContext`. It is deliberate — it stops the marketplace becoming a cold-contact list and means every thread a farmer receives already states its purpose. The UI therefore asks which requirement an approach is for, and explains the rule when the buyer has none, rather than failing at the API.

5. How it all connects — the dependency graph



Service layering. The four amber boxes are the shared engines — the reason both selling channels are directly comparable. `buyerComparisonService` reuses `marketService` wholesale rather than reimplementing mandi costing.

The hub services

Measured by inbound dependencies across `backend/src/services/`:

SERVICE	USED BY	CONSUMERS, AND WHAT THEY TAKE FROM IT
spoilageService	6	availability, buyerComparison, market, matching, requirement, sellTiming — crop profiles, shelf life, perishability, risk
marketPriceService	6	availability, marketPriceIngest, market, offer, pricePrediction, requirement — observed prices, freshness, date helpers
routingService	5	availability, buyerComparison, market, matching, transport — distance, detour factor, coordinate validation
netReturnService	3	buyerComparison, market, sellTiming — the expected-money ledger
transportCostService	3	buyerComparison, market, transport — freight for a load over a distance
availabilityService	4	buyerComparison, matching, offer, (controller) — the four-bucket quantity ledger

THE SINGLE MOST IMPORTANT INTERCONNECTION

`buyerComparisonService` → `marketService` → `netReturnService`. A marketplace comparison does not reimplement mandi costing — it *calls the mandi engine*. That is why `/selling-options` can rank a private buyer against an APMC mandi honestly: both numbers came out of the same function, with the same spoilage model, the same routing, and the same freight rates. Any change to how money is calculated automatically applies to both channels.

End-to-end: one farmer's path through the system

1. **Register and draw a field** — `farms` row with a boundary polygon. Coordinates unlock everything downstream.
2. **Watch crop vigour** — the NDVI job samples Sentinel-2 for that polygon daily and alerts on a drop.
3. **Photograph a suspect leaf** — the CNN classifies it, severity is scored, and the advisory is merged with weather-derived disease risk.
4. **Record the harvest** — a crop-availability listing, pre-filled from the saved field so only the kilograms are typed.
5. **Ask where to sell** — the mandi pipeline ranks nearby markets by expected money, itemising every deduction, and says whether to sell now or wait.
6. **Check the direct channel** — buyers wanting that crop are ranked by money in hand; `/selling-options` puts them beside the mandis.
7. **Negotiate** — private thread, structured offers, counter-offers.
8. **Agree** — acceptance transactionally reserves the crop, moving kilograms from `available` to `reserved` under a row lock.
9. **Arrange transport** — hauliers near the destination from Google Places, costed against the live diesel price.

Steps 5 and 6 are the same question asked of two channels, answered by one engine. That is the system's thesis in one sentence.

6. Data model — 28 tables

Schema files live in `database/schemas/` and are **fully idempotent** (every statement `IF NOT EXISTS`), applied automatically on each boot by `config/db.js`, in dependency order: `market.sql` → `marketplace.sql` → `transport_rates.sql` → `market_providers.sql`. A fresh clone therefore works with no manual `psql` step.

GROUP	TABLES	NOTES
Identity	<code>users</code> , <code>user_roles</code>	farmer implicit; buyer/admin granted
Fields	<code>farms</code>	Boundary polygon in JSONB; origin for all geo work
Crop health	<code>ndvi</code> , <code>ndvi_history</code> , <code>diseases</code> , <code>spoilage</code>	Unique index on (<code>field_id</code> , <code>captured_date</code>) stops duplicate NDVI history
Market data	<code>markets</code> , <code>market_prices</code> , <code>crop_profiles</code> , <code>price_predictions</code> , <code>market_recommendations</code> , <code>price_ingest_runs</code>	Every price row carries source + demo flag; recommendations stored with engine version
Provider mapping	<code>commodity_provider_map</code> , <code>market_provider_map</code>	Translates data.gov.in vocabulary to internal keys
Transport	<code>transport_config</code> , <code>transport_rate_quotes</code> , <code>fuel_prices</code>	Quotes have a max-age guard; diesel price drives per-km cost
Marketplace	<code>buyer_profiles</code> , <code>buyer_requirements</code> , <code>farmer_crop_availability</code> , <code>marketplace_conversations</code> , <code>marketplace_messages</code> , <code>marketplace_offers</code> , <code>marketplace_deals</code> , <code>marketplace_notifications</code> , <code>marketplace_reports</code>	Where the integrity constraints concentrate — see below
Advisory	<code>farm_ai_advice</code>	Cached plain-language narration

THE CONSTRAINTS THAT CARRY THE SAFETY GUARANTEES

- `CHECK (available_kg + reserved_kg + sold_kg <= total_harvested_kg)` — makes selling the same crop twice *structurally* impossible, not merely guarded in code.
- `UNIQUE (requirement_id, farmer_user_id, buyer_user_id)` on conversations — one thread per relationship, enforced by `ON CONFLICT` rather than a read-then-write race.
- `ON DELETE RESTRICT` on deals and offers — history cannot be erased from under an agreed deal. This dictates deletion order in the seed script's cleanup: deals, then offers, then requirements.
- `is_demo_data` flags throughout, so demo rows can be identified and purged regardless of owner.

7. Complete API surface

Mounted in `backend/src/app.js`. All marketplace and buyer routes require a real JWT.

BASE	ENDPOINTS
/api/auth	POST /register · POST /login
/api/farms	GET /user · GET /:id · POST / · POST /:farmId/ndvi · PUT /:id · DELETE /:id
/api/ndvi	POST /calculate · POST /timeseries · POST /advice · GET /history · GET /health/:ndviValue
/api/disease /api/diseases	GET /health · POST /detect · GET /farm/:farmId · POST /combined-advisory
/api/weather	GET /health · /current · /forecast · /hourly · /alerts · /disease-risk · /recommendations · /complete
/api/spoilage	GET /health · GET /options · POST /assess · POST /predict · GET /history/:farmId
/api/crops	GET / · GET /:crop — the 18-crop catalogue; what frontend pickers read
/api/market /api/markets same router, both paths	GET /health · POST /ingest · GET /provider/coverage · GET /prices · GET /forecast · GET /price-trend · GET /channels · GET /transport-options · POST /recommend · POST /explain · GET /history/:farmId · GET / · GET /:id · GET /:id/prices
/api/transport	GET /health · GET /search · GET /rate-basis · GET /fuel-price/history · POST /fuel-price · POST /quotes · POST /config/reload
/api/buyers	GET /me/roles · GET /me · PATCH /me · POST /me/verification · POST /profile · GET /admin/pending · POST /admin/:id/verification · POST /admin/:id/suspension · GET /:id (public projection)
/api/buyer-requirements	POST / · GET / (?mine=true) · GET /:id · PATCH /:id · POST /:id/publish · POST /:id/close · GET /:id/matching-farmers
/api/marketplace farmer side	GET POST /farmer/availability · PATCH DELETE /farmer/availability/:id · GET /farmer/availability/suggestions · GET /farmer/matches · GET /farmer/top-buyers · GET /selling-options
/api/marketplace buyer side	GET /availability (browse all listings) · GET /buyer/summary
/api/marketplace negotiation	POST GET /conversations · GET /conversations/:id/messages · POST /conversations/:id/messages · POST /conversations/:id/read · POST /conversations/:id/block · GET /messages/:id/attachment · POST GET /offers · GET /offers/:id · POST /offers/:id/{accept,reject,counter,withdraw} · GET /deals · GET /deals/:id · PATCH /deals/:id/status · GET /notifications · POST /notifications/read-all · POST /notifications/:id/read · POST /reports
ml-service :8000	GET /health · GET /model-info · POST /predict (disease) · POST /predict/price · GET /price-model/info
satellite :5001	GET /health · GET /api/status · POST /api/ndvi/calculate · POST /api/ndvi/timeseries · GET /api/ndvi/health/<v> · POST /api/satellite/{available,preview,mosaic}

8. Verified status right now

Live health checks and test runs, executed against the running stack on 4 October 2026.

CHECK	RESULT	DETAIL
Backend test suite	289 / 289	13 test files: money, netReturn, ranking, spoilage, sellTiming, routing, transport, transportCost, freightRate, priceProvider, marketValidator, marketplace, recommend integration
Frontend production build	CLEAN	573 modules transformed, no errors
/api/health	OK	Server is running
/api/market/health	OK	16 markets; real MANDI_API prices for 14 crops; all currently STALE (latest observation 2026-09-24)
/api/spoilage/health	OPERATIONAL	18 supported crops
/api/weather/health	OPERATIONAL	Open-Meteo, no API key required
/api/transport/health	OK	Google Places configured and enabled
/api/disease/health	ML DOWN	{backend:"healthy", ml_service:"unavailable"}
Buyer journey, 15 steps	ALL PASS	register → browse → add business → distances appear → draft → publish → edit → matches → chat → offer → mark read → profile edit → close
Authorization guards	HOLD	Cross-buyer access 403s; over-quantity rejected; self-awarded verification refused; blocked threads reject messages; no coordinate leak in browse payload

ACTION NEEDED — PORT 8000 CONFLICT

Disease detection and price forecasting are both unavailable for the same reason, and it is **not a code defect**. Port 8000 is currently held by an unrelated process titled "*LLM Gateway*" (its `/openapi.json` advertises `/api/chat`, `/api/skills` — nothing to do with AgriChain). The backend's `ML_SERVICE_URL` points at `127.0.0.1:8000` and reaches that instead. Either stop the conflicting process and start `ml-service` there, or run `ml-service` on another port and update `ML_SERVICE_URL`. Both modules degrade correctly in the meantime — nothing crashes.

9. What is remaining

Gaps in priority order

#	ITEM	DETAIL AND THE HONEST SIZE OF THE JOB
1	Price model trained on demo data	Metadata says <code>trained_on_real_data: false</code> . It beats a persistence baseline by 0.52% — effectively no signal. Needs retraining on the real MANDI_API history, which is accumulating but currently thin (8–68 observations per crop). This is a data problem before it is a modelling problem.
2	No Row Level Security	Unenforceable through a pooled superuser. Authorization is service-layer plus DB constraints. The largest architectural gap; migration path documented. Affects all nine modules.
3	Mandi price data is stale and thin	Real prices exist for 14 crops but the newest is 10 days old and some crops have under 15 observations. <code>POST /api/market/ingest</code> and <code>npm run ingest:prices</code> exist; what is missing is a scheduled job running them.
4	Satellite service not deployable as-is	Needs Google Earth Engine credentials and the Flask process running. NDVI is the one module with no offline fallback.
5	No frontend tests	Zero component tests; confidence rests on <code>vite build</code> plus backend contract tests. React Testing Library is the clearest addition. Note also that the UI has <i>never been verified in a real browser</i> in recent work — only compiled.
6	Chat is polling, not push	6 s, visibility-aware. Adequate for a negotiation; not what you would ship for a chat product. SSE or socket.io is the upgrade and <i>the service layer needs no change for it</i> .
7	Three dead farmer-side routes	<code>/marketplace/compare</code> (<code>MyCropsForSale.jsx:380</code> , <code>BuyerMarketplace.jsx:801</code>), <code>/marketplace/browse</code> (<code>BuyerMarketplace.jsx:759</code>), <code>/marketplace/requirements/:id</code> (<code>BuyerMarketplace.jsx:791</code>). All navigate to routes absent from <code>App.jsx</code> . The buyer-side equivalents were fixed; these need a farmer-facing requirement-detail and a comparison page.
8	Dead files	<code>backend/src/routes/recommendationRoutes.js</code> is empty and not mounted. <code>backend/src/models/User.js</code> and <code>NDVI.js</code> are 0 bytes. <code>frontend/src/services/{api,ndviService,spoilageService,diseaseService}.js</code> are all 0 bytes. Safe to delete; they mislead a reader into thinking an ORM layer exists.
9	Expiry is lazy, no scheduler	Offer and requirement expiry apply on read. Correct, but a nightly job is needed to notify buyers of <i>upcoming</i> expiry — the <code>REQUIREMENT_EXPIRING_SOON</code> event type exists and nothing emits it.
10	Mobile untested on device	Responsive breakpoints and bottom-sheet dialogs are in place but verified only at narrow viewport widths. The layout is desktop-first (fixed <code>m1-72</code> sidebar).

Deliberately out of scope

No payments, escrow or settlement · no driver tracking or vehicle GPS · no SMS or email notifications (in-app only) · no push notifications · buyer contact details intentionally withheld from farmers (change

`buyerService.toPublic()` if that is ever wanted) · no multi-language ML output (the Gemini narration layer handles language, and cannot alter figures).

CORRECTION TO AN OLDER DOCUMENT

`docs/Marketplace/README.md §14.5` states that the requirement-detail page is "not built" and that those routes "navigate to existing pages". **That is now out of date** — `MyRequirements.jsx` implements requirement detail with live matching farmers at `/buyer/requirements/:requirementId`. The buyer-detail page remains unbuilt.

10. Running the stack

```
# 1 · PostgreSQL must be running on 5432. Schema applies itself on boot.

# 2 · Backend (http://localhost:3000)
cd backend && npm install && npm run dev

# 3 · Frontend (http://localhost:5173)
cd frontend && npm install && npm run dev

# 4 · ML service (http://localhost:8000) – disease CNN + price model
cd ml-service && pip install -r requirements.txt
uvicorn api.app:app --port 8000          # free the port first, see §8

# 5 · Satellite service (http://localhost:5001) – needs Earth Engine creds
cd satellite-service && pip install -r requirements.txt && python app.py
```

USEFUL SCRIPTS

COMMAND	WHAT IT DOES
npm run seed:marketplace	Creates the 3-account demo scenario proving the price-vs-distance trade-off. Idempotent — clears prior demo data first. Add -- --clear to remove.
npm run seed:market	Markets and crop profiles
npm run seed:demo-prices	Synthetic price history (flagged DEMO_SEED)
npm run ingest:prices	Pulls real prices from data.gov.in
npm run map:markets	Maps provider market names to internal ids
npm test	289 tests. test:unit and test:integration split available.

DEMO CREDENTIALS

After `npm run seed:marketplace` — password `demo-only-not-secure` for all three:
`demo-farmer@example.test` · `demo-buyer-a@example.test` (verified processor, 135 km, ₹31/kg) · `demo-buyer-b@example.test` (unverified wholesaler, 16 km, ₹28/kg, collects)
Only `ORS_API_KEY` is unset in `backend/.env`, and it is optional — OSRM is the default router.

11. Glossary

TERM	MEANING IN THIS CODEBASE
APMC / mandi	Agricultural Produce Market Committee — the regulated wholesale market channel
Expected money	What the farmer keeps after spoilage, freight and selling costs. The only ranking key.
Net realizable return	Formal name for the same figure; hence <code>netReturnService</code>
Hamali	Manual loading/unloading charge at a mandi, levied per quintal
Cess	Statutory market fee, a percentage of realised value
Quintal	100 kg. Indian mandi prices are quoted per quintal — the most common data-entry error in this domain, which is why price fields warn "per kilogram, not per quintal".
Modal price	The most frequently traded price on a given day; what the forecast targets
NDVI	Normalised Difference Vegetation Index — crop vigour from satellite near-infrared and red bands
Q10 model	Respiration roughly doubles per 10 °C above optimum; the spoilage engine's temperature factor
Requirement	A buyer's posted demand: crop, quantity, price, date, delivery terms
Availability	A farmer's posted supply, tracked across four quantity buckets
Offer	A structured proposal from either side. Nothing is agreed until accepted.
Deal	An accepted offer. Reserves crop; has its own fulfilment status lifecycle.
DEMO_SEED	Source label for synthetic data. Always paired with <code>is_demo_data = TRUE</code> .
MANDI_API	Source label for real prices ingested from <code>data.gov.in</code>
RULE_BASED_BASELINE	Provenance label asserting a deterministic rule set, not a fitted model
STRAIGHT_LINE	Distance is haversine, not a road route. Never presented as the latter.

AgriChain System Handoff · compiled 4 October 2026 from the live codebase, the running database and executed HTTP calls. Figures quoted (289 tests, 573 modules, 98.58% model accuracy, 16 markets, 28 tables, 18 crops) were each read from the system rather than from prior documentation. Where this document and an older module README disagree, this one is newer — see the correction note in §9.